Discipline : Enseignement de spécialité NSI

Les arbres binaires

Contenus : Arbres : structures hiérarchiques. Arbres binaires : nœuds, racines, feuilles, sous-arbres gauches, sous-arbres droits. Récursivité. Algorithmes sur les arbres binaires et sur les arbres binaires de recherche.

Capacités attendues: Identifier des situations nécessitant une structure de données arborescente.
Évaluer quelques mesures des arbres binaires (taille, encadrement de la hauteur, etc.).
Écrire un programme récursif. Analyser le fonctionnement d'un programme récursif.
Calculer la taille et la hauteur d'un arbre. Parcourir un arbre de différentes façons (ordres infixe, préfixe ou suffixe ; ordre en largeur d'abord).
Rechercher une clé dans un arbre de recherche, insérer une clé.

Pré-requis : Langage python, POO

Vous ferez une copie d'écran de <u>vos résultats</u> pour toutes les questions du TP.

A Programmation de la classe arbre

On donne le fichier de départ de la classe arbre contenant des méthodes pour afficher graphiquement les arbres avec le module matplotlib de Python. La classe PileFile utile pour ce TP est fournie.

<u>A-1 (/3pts) Taille d'un arbre</u>

Créer une méthode de la classe arbre permettant de déterminer le nombre de nœuds en vous aidant du pseudo code suivant :

```
méthode taille(arbre):
        si arbre est vide alors
                retourner 0
        sinon
                taille_gauche = taille(sous_arbre_gauche)
                taille_droite = taille(sous_arbre_droit)
                retourner(1+taille_gauche+taille_droite)
```

<u>A-2 (/2pts) Hauteur d'un arbre</u>

Créer une méthode de la classe arbre permettant de déterminer la hauteur d'un arbre en vous aidant du pseudo code suivant et de la méthode définie précédemment:

```
méthode hauteur(arbre):
        si arbre est vide alors
                retourner -1
        sinon
                hauteur_gauche = hauteur(sous_arbre_gauche)
                hauteur_droite = hauteur(sous_arbre_droite)
                retourner(1+max(hauteur_gauche,hauteur_droite))
```

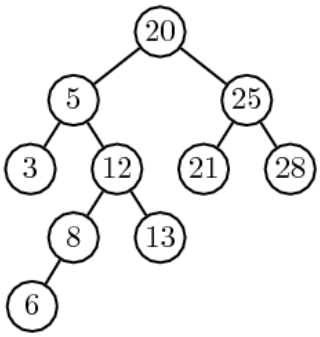
<u>A-3 (/1pt) Test sur arbre</u>

Créer une méthode estVide() de la classe arbre permettant de déterminer si l'arbre pris en argument est vide.

A-4 (/3pts) Tests unitaires

Tester ces méthodes dans un programme principal main.py en n'oubliant pas les imports pour les arbres suivants et donner les résultats obtenus :

a= [5,[3,[4,[2,[],[]],[1,[],[]]],[6,[7,[10,[],[]],[11,[],[]]],[8,[],[]]]],[2,[],[]]]
b= [1,[2,[4,[8,[],[]],[9,[],[]]],[5,[10,[],[]],[]]],[3,[6,[],[]],[7,[],[]]]]
c=[2, [1, [0, [], []], []], [5, [4, [3, [], []], []], [12,[9, [8, [6, [], [7, [], []]], []], [10, [], [11, [], []]]], [13, [], [14, [], [18, [17, [15, [], [16, [], []]],[]], [19, [], []]]]]]]]

d = 

e= proposer une liste d'élément (prénom nom) représentant votre généalogique partiel dont vous avez connaissance puis appliquer les méthodes taille et hauteur à cet arbre.
Insérer dans votre compte –rendu le graphique associé à chaque arbre avec la méthode dessiner().

B Algorithme de recherche dans un arbre

B-1 (/3pts) Algorithme de recherche en largeur d'abord

On désire programmer un algorithme de parcours de l'arbre en largeur c'est-à-dire que l'on explore un niveau complètement avant de passer au niveau suivant. Pour y arriver, on utilisera un objet de la classe PileFile utilisé en File programmée dans le TP précédent.
Programmer une méthode de la classe arbre en vous aidant du pseudo code suivant :

```
méthode parcours_largeur(arbre)
        visite = créer file fifo
        lst=créer liste vide
        ajouter arbre à visite (enfiler)
        tant que visite n'est pas vide
                nœud reçoit le premier élément de visite (defiler)
                si nœud est une liste
                        ajouter à la fin de lst le premier élément de nœud
                        si fils gauche non vide
                                ajouter fils gauche à visite (enfiler)
                        si fils droit non vide
                                ajouter fils droit à visite (enfiler)
                sinon
                        ajouter à la fin de lst noeud
        retourner lst
```

Tester cette méthode dans un programme principal pour les arbres précédents et donner les résultats obtenus. Pour analyser ce programme, on affichera l'état de la file lors de son exécution. (copies d'écran du résultat demandé)

B-2 (/4pts) Algorithme de recherche en profondeur d'abord

On désire programmer un algorithme de parcours de l'arbre en profondeur c'est-à-dire que l'on explore une branche de l'arbre complètement avant de passer à la branche suivante de la gauche vers la droite. Pour y arriver, on utilisera un objet de la classe PileFile utilisé en Pile programmée dans le TP précédent.
Le parcours sera de type préfixe par défaut et non récursif c'est-à-dire parcours de la racine, fils gauche puis fils droit. On utilisera une liste pour stocker les éléments correspondant aux valeurs des différentes feuilles de l'arbre.
Programmer une méthode de la classe arbre en vous aidant du pseudo code suivant :

```
méthode parcours_profondeur(arbre)
        visite = créer pile lifo
        lst1=créer liste vide
        ajouter arbre à visite (empiler)
        tant que visite n'est pas vide
                noeud = le dernier élément de visite (dépiler)
                si nœud est une liste
                        ajouter à la fin de lst1 le premier élément de nœud
                        si fils droit non vide
                                ajouter fils droit à visite (empiler)
                        si fils gauche non vide
                                ajouter fils gauche à visite (empiler)
                sinon
                        ajouter à la fin de lst1 noeud
        retourner lst1
```

Tester cette méthode dans un programme principal pour les arbres précédents et donner les résultats obtenus. Pour analyser ce programme, on affichera l'état de la pile lors de son exécution. (copies d'écran du résultat demandé)

B-3 Algorithme de recherche en profondeur d'abord (récursif)

B-3-1 (/3pts) Algorithme de recherche en profondeur d'abord prefixe

```
méthode parcours_profondeur_recursif_prefixe(arbre):
        lst2=créer liste vide
        si nœud est une liste
                ajouter à la fin de lst2 le premier élément de nœud
                si fils gauche non vide
                        parcours_profondeur_recursif_prefixe(fils gauche)
                si fils droit non vide
                        parcours_profondeur_recursif_prefixe(fils droit)
        sinon
                ajouter à la fin de lst2 nœud
        retourner lst2
```

Pour analyser la récursivité de ce programme on affichera les listes intermédiaires générées lors de son exécution. (copies d'écran du résultat demandé)

B-3-2 (/1pt) Algorithme de recherche en profondeur d'abord infixe

```
méthode parcours_profondeur_recursif_infixe(arbre):
        lst2=créer liste vide
        si nœud est une liste
                si fils gauche non vide
                        parcours_profondeur_recursif_infixe(fils gauche)
                ajouter à la fin de lst2 le premier élément de nœud
                si fils droit non vide
                        parcours_profondeur_recursif_infixe(fils droit)
        sinon
                ajouter à la fin de lst2 nœud
        retourner lst2
```

B-3-3 (/1pt) Algorithme de recherche en profondeur d'abord postfixe

```
méthode parcours_profondeur_recursif_postfixe(arbre):
        lst2=créer liste vide
        si nœud est une liste
                si fils gauche non vide
                        parcours_profondeur_recursif_postfixe(fils gauche)
                si fils droit non vide
                        parcours_profondeur_recursif_postfixe(fils droit)
                ajouter à la fin de lst2 le premier élément de nœud
        sinon
                ajouter à la fin de lst2 nœud
        retourner lst2
```

Tester ces méthodes dans un programme principal pour les arbres précédents et donner les résultats obtenus.